Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

HWSW-Systeme

VHDL-Übung 08: Datentypen und Datenspeicher

M.Sc. Eike Trumann, M.Sc. Jasper Homann,

Prof. Dr.-Ing. Guillermo Payá Vayá

12. Dezember 2025

Einführung

Ergänzend zu den Folien aus der Vorlesung sollte bei diesen Aufgaben folgende Tabelle, welche direkt aus dem Anhang zum VHDL-2008-Standard stammt beachtet werden:

Operation	Result Range		
A + B	Max(A'left, B'left) + 1	downto	Min(A'right, B'right)
A - B	Max(A'left, B'left) + 1	downto	Min(A'right, B'right)
A * B	A'left + B'left + 1	downto	A'right + B'right
A rem B	Min(A'left, B'left)	downto	Min(A'right, B'right)
Signed A/B	A'left - B'right + 1	downto	A'right - B'left
Signed A mod B	Min(A'left, B'left)	downto	Min(A'right, B'right)
Signed reciprocal(A)	-A'right	downto	-A'left - 1
abs A	A'left + 1	downto	A'right
- A	A'left + 1	downto	A'right
Unsigned A/B	A'left - B'right	downto	A'right - B'left - 1
Unsigned A mod B	B'left	downto	Min(A'right, B'right)
Unsigned reciprocal(A)	-A'right + 1	downto	-A'left

Aufgabe VHDL-800: *Navigationssystem*

Für unsere Freunde in Amerika muss ein Navigationssystem angepasst werden. Während das Navigationssystem in Europa seine Strecken in Meter angibt, verlangen amerikanische Nutzer nach einer Angabe in Meilen. Eine Schwierigkeit, die während der Umrechnung zu beachten ist, ist, dass in Meter eine Angabe als Ganzzahl zweckmäßig ist, während bei der Nutzung in Meilen eine Angabe als Festkommazahl mit 10 binären Nachkommastellen zweckmäßig ist.

VHDL-800

Eine der in Amerika verwendeten Meilen ist exakt 1609,344 m lang. Da die amerikanischen Kunden Wert auf eine korrekte Benutzung ihres Maßsystems legen, muss dieser Wert exakt verwendet werden, es darf also kein Rundungsfehler bei diesem Wert auftreten.

```
entity navigation_adapter is
port (
meters_i  : IN unsigned(25 downto 0);
miles_o : OUT ufixed(15 downto -10)
);
end entity navigation_adapter;

architecture rtl of navigation_adapter is
  -- TODO
begin
 -- TODO
end rtl;
```

Aufgabe VHDL-801: *Tankstelle*

Um Hackerangriffe auf eine Zapfsäule zu verhindern, soll eine Recheneinheit für den Benzinpreis in Hardware implementiert werden. Um die Entwicklung der Hardwareeinheit zu vereinfachen, wurde ein Durchflussmesser an der Zapfsäule eingebaut, der die Menge in Litern mit 10 binären Nachkommastellen ausgeben kann. Wie viele Tankstellen, gibt unser Auftraggeber seine Preise nicht nur in Euro und Cent an, sondern verwendet Beträge hinter der Centstelle. Um diese hinreichend genau darstellen zu können, wurde vereinbart, dass der Preis als Ganzzahl mit einer Genauigkeit von 1/10 ct übergeben wird. Die Kasse wird den Endbetrag auf Euro und Cents runden, benötigt allerdings eine Festkommazahl in Euro als Eingabe.

VHDL-801

```
entity zapfsaeule is
port (
  liters_i : IN ufixed(9 downto -10);
  price_i  : IN unsigned(15 downto 0);
  total_o  : OUT ufixed(12 downto -10)
);
end entity zapfsaeule;
```

```
architecture rtl of zapfsaeule is
  -- TODO
begin
 -- TODO
end rtl;
```

Aufgabe VHDL-802: *Universaladapter*

Im vom EIS angebotenen Hardwarepraktikum verwenden verschiedene Gruppen häufig unterschiedliche Datentypen. Um dieses Chaos zusammenzubauen, wird ein Adapter benötigt, der die entsprechenden Datentypen ineinander umwandeln kann.

Die Verbindungen sollen für folgende Anwendungen benutzt werden:

VHDL-802

a) Teilnehmerstatistik Drei Gruppen arbeiten an einer Teilnehmerstatistik für das Labor. Eine Gruppe stellt hierbei die Teilnehmeranzahl des aktuellen Labors als unsigned-Wert in einem `std_ulogic_vector` zur Verfügung. Eine Gruppe verwaltet die Vorjahresstatistiken und stellt ebenfalls eine Teilnehmerzahl als unsigned-Wert in einem `std_ulogic_vector` zur Verfügung. Die dritte Gruppe benötigt für die Ausgabe jedoch die Differenz von beiden, die als signed-Wert in einem `std_ulogic_vector` dargestellt wird.

b) Labordauer Das Labor sollte normalerweise zwei mal 90 Minuten dauern. Eine Gruppe möchte feststellen, ob diese Zeit eingehalten wird. Als Eingangswert wird hierbei die Start- und Endzeit von einer Systemuhr verwendet, die einen unsigned 64-Bit-Wert in Sekunden übergibt. Der Ausgang soll in Minuten (abgerundet) die Dauer der Veranstaltung angeben.

c) Mehrkosten Da es manchmal im Labor länger dauert, bis die letzte Gruppe fertig wird oder das Labor früher beendet werden kann, weil alle Gruppen bereits fertig sind, wird zusätzliche Arbeitszeit der Mitarbeiter verbraucht oder freigegeben. Die Zeitdifferenz ist hierbei ein signed-Wert in Minuten, bei dem negative Zahlen zusätzliche Minuten und positive Zahlen gesparte Minuten angeben, die Anzahl der Mitarbeiter ein unsigned-Wert und der Ausgang ein signed-Wert der insgesamt zusätzlich verbrauchten Minuten (negativ), bzw. der gesparten Minuten (positiv) sein.

d) Dateigröße Eine Gruppe gibt die Größe von gespeicherten Daten als Ganzzahl in Kibibytes (= 1024 Byte) an. Die andere Gruppe verwendet Mebibyte (= 1024 KiB), allerdings mit 10 binären Nachkommastellen. Der Informationsgehalt ist somit exakt gleich und es darf kein Rundungsfehler auftreten.

```
entity universal_adapter is
port (

-- a) Teilnehmerstatistik
-- Die Teilnehmerzahlen am Eingang enthalten unsigned-Werte.
```

```
-- Der Ausgang muss als signed-Wert die Differenz der Teilnehmerwerte angeben.
teilnehmerzahl_aktuell_i : IN std_ulogic_vector(7 downto 0);
teilnehmerzahl_vorjahr_i : IN std_ulogic_vector(7 downto 0);
teilnehmerzahl_differenz_o : OUT std_ulogic_vector(8 downto 0);

-- b) Labordauer
-- Alle Werte sind unsigned
startzeit_sekunden_i : IN std_ulogic_vector(63 downto 0);
endzeit_sekunden_i: IN std_ulogic_vector(63 downto 0);
dauer_minuten_o: OUT std_ulogic_vector(9 downto 0);

-- c) Mehrkosten
-- Die Zeitdifferenz ist ein signed-Wert
zeitdifferenz_i: IN std_ulogic_vector(8 downto 0);
-- die Personenzahl ist unsigned
personen_i: IN std_ulogic_vector(3 downto 0);
gesamtaufwand_o: OUT std_ulogic_vector(20 downto 0);

-- d) Dateigröße
-- Hier müssen Kibibytes in Mebibytes umgerechnet werden.
-- Durch die Nachkommastellen darf hierbei keine Information verloren gehen.
KiB_i : IN std_ulogic_vector(29 downto 0);
MiB_o : OUT ufixed(19 downto -10)

);
end entity universal_adapter;
```

Aufgabe VHDL-803: *Schaltplan (aus Klausur WiSe2022/2023)*

Gegeben ist folgende Schaltung. Arithmetik wird mit signed Werten durchgeführt und Überläufe werden abgeschnitten.

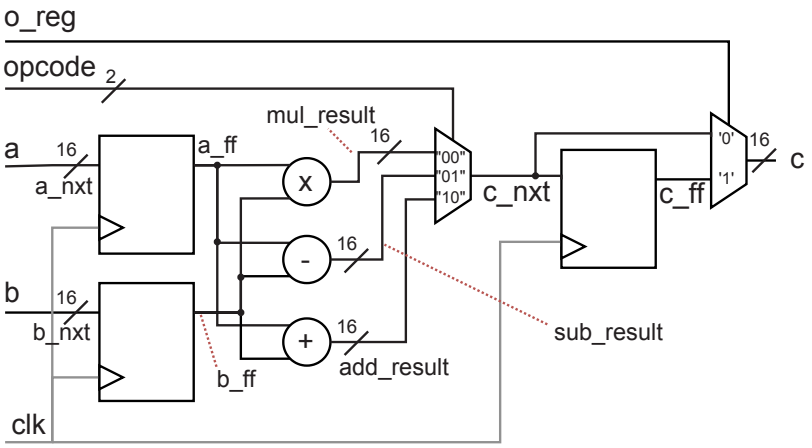
VHDL-803

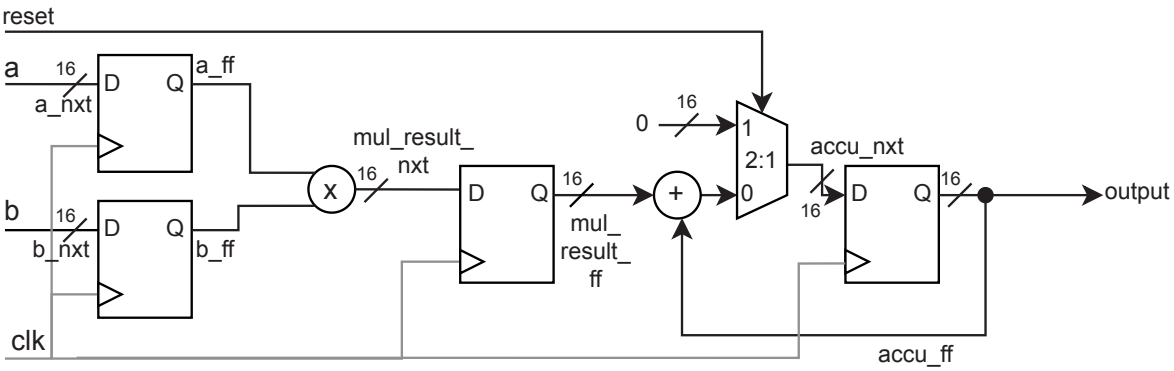
Ergänzen sie das VHDL-Template an den vorgesehenen Stellen, um diese Funktionalität zu implementieren.

Aufgabe VHDL-804: *Schaltplan (aus Klausur SoSe 2023)*

Gegeben ist folgende Schaltung. Arithmetik wird mit signed Werten durchgeführt und Überläufe werden abgeschnitten.

Implementieren sie die Funktionalität in das VHDL-Template.


o_reg
opcode
2
a
16
a_nxt
a_ff
mul_result
16
x
"00"
"01"
"10"
c_nxt
c_ff
'0'
'1'
16
c
-
16
b
16
b_nxt
+
16
add_result
sub_result
b_ff
clk


reset
a
16
a_nxt
D
Q
a_ff
mul_result_
nxt
16
x
0
16
1
2:1
0
accu_nxt
16
b
16
b_nxt
D
Q
b_ff
D
Q
16
mul_
result_
ff
+
16
D
Q
16
output
clk
accu_ff

VHDL-804